

Relatório Desafio Fácil, Médio e Difícil: Algoritmos de Ordenação (Selection Sort, Bubble Sort e Insertion Sort)

Nome: Paulo Willian Costa Rodrigues

Curso: Sistemas de Informação

Disciplina: Algoritmos e Complexidade

Professor: Alexandre Freitas

1. Introdução

Este relatório descreve o funcionamento de um programa Python que implementa e compara três algoritmos de ordenação: Selection Sort, Bubble Sort e Insertion Sort.

2. Descrição do Código – Desafio Fácil

Link Código Colab:

<https://colab.research.google.com/drive/1RRQAo0SomfoojyUx7pSZziSuJ0TlrR16?usp=sharing>

O programa é dividido em quatro partes principais:

2.1 Geração do Array Aleatório

Um array de 10 elementos é criado, onde cada elemento é um número inteiro aleatório entre 1 e 50.

A biblioteca random é utilizada para gerar esses números.

```
array = [random.randint(1, 50) for _ in range(10)]
```

2.2 Algoritmos de Ordenação

I. Selection Sort

O algoritmo percorre o array procurando o menor elemento e o coloca na primeira posição.

Depois procura o segundo menor e o coloca na segunda posição, e assim sucessivamente.

```
# Função Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

II. Bubble Sort

Compara elementos adjacentes e os troca de posição se estiverem na ordem errada.

Repetidamente "borbulha" o maior elemento para o final do array.

```
# Função Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

III. Insertion Sort

Constroi o array final ordenado um item por vez.

Para cada elemento, percorre a parte ordenada do array e insere o elemento na posição correta.

```
# Função Insertion Sort
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

2.3 Execução dos Algoritmos

O array aleatório gerado é copiado para três novas variáveis (array_selection, array_bubble e array_insertion) para que cada algoritmo trabalhe sobre os mesmos dados iniciais.

Após cada ordenação, o array ordenado é impresso no terminal.

```
# Gerar array aleatório
array = [random.randint(1, 50) for _ in range(10)]
print("Array original:", array)

# Ordenando com Selection Sort
array_selection = array.copy()
print("Selection Sort:", selection_sort(array_selection))

# Ordenando com Bubble Sort
array_bubble = array.copy()
print("Bubble Sort:", bubble_sort(array_bubble))

# Ordenando com Insertion Sort
array_insertion = array.copy()
print("Insertion Sort:", insertion_sort(array_insertion))
```

3. Resultado Esperado

O programa exibirá inicialmente o array original desordenado.

Em seguida, mostrará três versões ordenadas do array, cada uma correspondente a um dos algoritmos aplicados.

O conteúdo ordenado deve ser o mesmo para todos os algoritmos, já que todos têm o objetivo de colocar os números em ordem crescente.

Exemplo de saída:

```
Array original: [36, 45, 40, 8, 46, 25, 33, 47, 37, 10]
Selection Sort: [8, 10, 25, 33, 36, 37, 40, 45, 46, 47]
Bubble Sort: [8, 10, 25, 33, 36, 37, 40, 45, 46, 47]
Insertion Sort: [8, 10, 25, 33, 36, 37, 40, 45, 46, 47]
```

4. Descrição do código – Desafio Médio

Neste segundo desafio, os algoritmos são adaptados para ordenar os dados em ordem decrescente a medir o tempo de execução para diferentes tamanhos de arrays.

Link Código Colab:

https://colab.research.google.com/drive/1LmOum1G4ad8XR5UmxO5Viblg5H-f_Euo?usp=sharing

4.1 Alterações nos Algoritmos

Os algoritmos agora comparam de forma invertida (> em vez de <) para ordenar do maior para o menor.

Exemplo:

```
if arr[j] > arr[max_idx]: # Alteração para ordem decrescente
```

4.2 Medição do Tempo

A função medir_tempo utiliza o módulo time para calcular a duração da execução de cada algoritmo.

```
# Função para medir o tempo de execução
def medir_tempo(algoritmo, arr):
    inicio = time.time()
    algoritmo(arr)
    fim = time.time()
    return fim - inicio
```

4.3 Tamanhos de Teste

Os testes foram realizados em arrays com 10, 100 e 1000 elementos com valores aleatórios entre 1 e 1000.

```
Array com 10 elementos:
Tempo do Selection Sort: 0.000010 segundos
Array ordenado (Selection Sort): [939, 936, 692, 663, 406, 384, 328, 181, 134, 92]
Tempo do Bubble Sort: 0.000011 segundos
Array ordenado (Bubble Sort): [939, 936, 692, 663, 406, 384, 328, 181, 134, 92]
Tempo do Insertion Sort: 0.000008 segundos
Array ordenado (Insertion Sort): [939, 936, 692, 663, 406, 384, 328, 181, 134, 92]
```

4.4 Análise de Resultados

O tempo aumenta conforme o tamanho do array cresce.

Insertion Sort foi ligeiramente mais rápido para arrays pequenos.

5. Descrição do Código – Desafio Difícil

Neste terceiro desafio, o foco é avaliar o desempenho dos algoritmos de ordenação com diferentes situações de entrada: Totalmente desordenado, já ordenado e parcialmente ordenado. Os testes são realizados em três tamanhos de arrays: 10, 100 e 1000 elementos.

Link Código Colab:

<https://colab.research.google.com/drive/1tO4Y9mNBlwaWWsvKS4fWYOAODWj6F XuW?usp=sharing>

5.1 Tipos de Arrays Utilizados

I. Totalmente Desordenado

Os dados são gerados aleatoriamente com números de 1 e 1000.

É a situação padrão para teste de algoritmos, pois representam um caso comum do mundo real.

```
# Totalmente desordenado
array_desordenado = [random.randint(1, 1000) for _ in range(tamanho)]
print("\nArray totalmente desordenado:")
print(array_desordenado)
array_selection = array_desordenado.copy()
tempo_selection = medir_tempo(selection_sort, array_selection)
print(f"Selection Sort: {tempo_selection:.6f} segundos")

array_bubble = array_desordenado.copy()
tempo_bubble = medir_tempo(bubble_sort, array_bubble)
print(f"Bubble Sort: {tempo_bubble:.6f} segundos")

array_insertion = array_desordenado.copy()
tempo_insertion = medir_tempo(insertion_sort, array_insertion)
print(f"Insertion Sort: {tempo_insertion:.6f} segundos")
```

II. Já Ordenado

O array aleatório é ordenado previamente usando a função sorted (...reverse = True).

Serve para testar se o algoritmo consegue identificar se não há necessidade de muitas trocas.

```

# Já ordenado
array_ordenado = sorted(array_desordenado, reverse=True)
print("\nArray já ordenado:")
print(array_ordenado)
array_selection = array_ordenado.copy()
tempo_selection = medir_tempo(selection_sort, array_selection)
print(f"Selection Sort: {tempo_selection:.6f} segundos")

array_bubble = array_ordenado.copy()
tempo_bubble = medir_tempo(bubble_sort, array_bubble)
print(f"Bubble Sort: {tempo_bubble:.6f} segundos")

array_insertion = array_ordenado.copy()
tempo_insertion = medir_tempo(insertion_sort, array_insertion)
print(f"Insertion Sort: {tempo_insertion:.6f} segundos")

```

III. Parcialmente Ordenado

Os cinco primeiros elementos do array ordenado são embaralhados com `Random.shuffle()`.

Representa um cenário intermediário, útil para avaliar o comportamento dos algoritmos em situações realistas onde a ordenação está quase correta.

```

# Parcialmente ordenado (5 primeiros fora de ordem)
array_parcialmente_ordenado = array_ordenado.copy()
random.shuffle(array_parcialmente_ordenado[:5]) # Embaralha os 5 primeiros
print("\nArray parcialmente ordenado (5 primeiros fora de ordem):")
print(array_parcialmente_ordenado)
array_selection = array_parcialmente_ordenado.copy()
tempo_selection = medir_tempo(selection_sort, array_selection)
print(f"Selection Sort: {tempo_selection:.6f} segundos")

array_bubble = array_parcialmente_ordenado.copy()
tempo_bubble = medir_tempo(bubble_sort, array_bubble)
print(f"Bubble Sort: {tempo_bubble:.6f} segundos")

array_insertion = array_parcialmente_ordenado.copy()
tempo_insertion = medir_tempo(insertion_sort, array_insertion)
print(f"Insertion Sort: {tempo_insertion:.6f} segundos")

```

5.2 Desempenho

Para cada tipo de array e tamanho, o tempo de execução é medido usando a função `time.time()` antes e depois da chamada do algoritmo antes e depois da chamada do algoritmo.

Isso permite comparar o desempenho relativo de cada algoritmo em diferentes cenários

Exemplo:

```
Testando com arrays de tamanho 10:  
  
Array totalmente desordenado:  
[772, 792, 865, 994, 767, 725, 142, 914, 443, 957]  
Selection Sort: 0.000014 segundos  
Bubble Sort: 0.000015 segundos  
Insertion Sort: 0.000010 segundos  
  
Array já ordenado:  
[994, 957, 914, 865, 792, 772, 767, 725, 443, 142]  
Selection Sort: 0.000009 segundos  
Bubble Sort: 0.000009 segundos  
Insertion Sort: 0.000003 segundos  
  
Array parcialmente ordenado (5 primeiros fora de ordem):  
[994, 957, 914, 865, 792, 772, 767, 725, 443, 142]  
Selection Sort: 0.000007 segundos  
Bubble Sort: 0.000008 segundos  
Insertion Sort: 0.000002 segundos
```

6. Conclusão

Insertion Sort é o mais eficiente nos casos já ordenados e parcialmente ordenados, especialmente em arrays pequenos e médios

Bubble Sort e Selection Sort são mais lentos, principalmente com arrays maiores ou totalmente desordenados.

É importante saber escolher o algoritmo certo dependendo da situação real do conjunto de dados.